goxpyriment — Timing Tests

christophe@pallier.org

Contents

Timing tests **1**
Part 1 — Measuring your own machine 1
Part 2 — What we found 13
Related tests 18
Contributing a report 18

Timing tests

This document does two things.

Part 1 shows you how to characterise the timing behaviour of *your* machine with the bundled `Timing-Tests` program — both the checks that need no equipment and the photodiode/TTL measurement that is the real answer — and how to read each of them.

Part 2 summarises what we have measured on our own machines, so you have reference points to compare against. Every figure there names the machine, the configuration, the date and the sample size. Nothing is quoted that we cannot point at a capture for.

> **The one-sentence version of Part 2:** on Linux the display stack dominates everything else by more than an order of magnitude — a compositor costs about sixteen times the onset jitter of a bare KMS/DRM console, and no amount of care in the experiment code recovers it.

Companion documents. `tests/Timing-Tests/README.md` is the flag-by-flag reference — invocations, defaults, equipment, wiring. This page explains what the tests *mean*. For the EEG/MEG question specifically, see Minimising trigger-to-stimulus jitter, and for the comparison against published packages, the mega-study comparison.

Part 1 — Measuring your own machine

Read this before you trust any number

This suite produces **two kinds of output, and they are not equally trustworthy.** Keeping them apart is the single most important thing on this page.

- **What the program prints to the console** — frame intervals, bright-phase durations, drain times, SOAs — comes entirely from software timestamps, in all six sub-tests without exception. It records when goxpyriment *believes* a flip happened, not what reached the panel.
- **What an instrument records while `av` runs** — a photodiode on the screen, the TTL line on a second channel, a microphone if you are checking audio — is a measurement of the physical event. This is ground truth.

So the suite *does* measure your hardware properly: that is what `av` is for, and `run-timing-tests.sh` can wrap those steps in a `bbtk-capture` recording so the stimulus always falls inside the capture window. What it cannot do is tell you any of it from the console. **Never read the printed statistics as if they were the instrument's.**

This is not a theoretical caveat. A presentation bug on a GNOME/Wayland machine once left the display showing stale frames for *seconds at a time* while the program's own numbers stayed textbook-perfect: bright-phase duration 199.915 ± 0.16 ms against a 200 ms target. Nothing in the console output hinted at a problem. It was visible only with a photodiode. (The cause and fix are in `apparatus/CLAUDE.md`; the regression test is `tests/test_clear_only_frames`.)

None of which makes the console output useless — it is the right tool for a different job. Software statistics genuinely detect dropped frames, scheduler jitter and audio-buffer effects, and `check`, `display` and `latency` need no equipment at all, so run them first. They just cannot confirm that light changed when you said it would.

If you are validating a rig for publication, take the hardware path: `av` plus a photodiode and a trigger box, then `timing-drift` to check the two against each other.

What the display actually does

Most monitors refresh at a fixed rate — 60 Hz (a new frame every 16.67 ms), 120 Hz, or 144 Hz. The driver synchronises presentation to that cycle (VSYNC), with two consequences:

- **Durations are quantised to multiples of the frame period.** At 60 Hz you can show a stimulus for 16.67, 33.33 or 50 ms — but not 25 ms. Plan accordingly, or see the `vrr` test.
- **Onset is not when you call the flip; it is the next VSYNC**, anywhere from 0 to 16.67 ms later. goxpyriment holds that offset constant once the pipeline is warm, which is why the first frames must be excluded — `-warmup` does this.

The screen does not change all at once An LCD paints top to bottom, so a square at the bottom of the screen lights measurably later than one at the top — on a 60 Hz panel, approaching a full frame period. A stimulus at screen centre appears roughly half a frame after one at the top.

This matters more than it sounds: if your photodiode sits in a corner but your stimulus is central, your measured onset carries a systematic bias larger than most of the jitter people worry about.

The gradient is a property of your panel, and you have to measure it. The `av` test draws five squares — four corners plus centre — precisely so you can. Put a photodiode on a top square and another on a bottom one; the difference is your scan-out gradient. Two squares on the same row are only microseconds apart and serve as a sanity check. Each display needs its own figure.

The six tests

```
No hardware needed
  check    Go/no-go: can this machine display and make noise at all?
  display  What is my true refresh rate, and how stable are frame intervals?
  latency  How long does SDL hold audio before it sounds?

Photodiode and/or trigger box
  av       THE stimulus timing test — visual + audio + TTL, every cycle
  vrr      Can I present arbitrary durations, not just multiples of a frame?
  rt       How precise is my reaction-time measurement?
```

`av` is the default: running with no `-test` flag runs it.

Test	What it measures	Reads its answer from
`check`	Nothing. Flashes white, plays a buzzer and a ping.	Your eyes and ears
`display`	Frame-interval distribution over `-duration-s`; estimated true refresh rate; the frame-pacing verdict	Console
`latency`	How long SDL holds PCM beyond a tone's nominal duration — the software audio pipeline delay	Console
`av`	Bright-phase duration, cycle period, software-side audio-vs-visual SOA — **and drives the photodiode/TTL capture that is the real measurement**	A recorder
`vrr`	Achieved-vs-target duration across a 1 ms-step sweep with VSync off	Console, then a photodiode
`rt`	SDL3 hardware event timestamp against the flip timestamp	Console

Note that `-test latency` labels its own output `drain:` and writes `test=drain` into the

info file — the drain time *is* the measurement.

Two related tests live in their own directories: `tests/test_gv_sync` (`.gv` playback synchronisation) and `tests/test_dlpio8` (DLP-IO8-G square-wave characterisation).

Build it once:

```
go build -o Timing-Tests ./tests/Timing-Tests
```

Run every test fullscreen — that is the default. `-w` exists for debugging the tests themselves; a windowed run measures your compositor, not goxpyriment.

Recording the machine (`-sysinfo`)

Before running anything, capture a snapshot. `-sysinfo` prints it and exits — it opens no window, though it initialises SDL briefly to enumerate displays and audio devices:

```
Timing-Tests -sysinfo > sysinfo-$(hostname)-$(date +%Y%m%d).txt
```

A real capture (`tests/Timing-Tests/report-dell-precision-5490-laptop-kmsdrm/sysinfo.txt`):

```
Machine:    product: Precision 5490  System: Dell Inc.  Type: laptop
System:     Host: is158520  Kernel: 7.0.0-29-generic x86_64  Uptime: 32m
            OS: Ubuntu 26.04 LTS  Shell: bash 5.3.9
CPU:        Model: Intel(R) Core(TM) Ultra 7 165H  Info: 16 cores / 22
 ↪  threads  Speed: 1076 MHz (min: 400 / max: 4700)
Memory:     RAM: total: 30.04 GiB  used: 2.12 GiB (7.0%)  Swap: total: 8.00
 ↪  GiB  used: 0 KiB (0.0%)
Graphics:   Card: Intel Corporation Meteor Lake-P [Intel Arc Graphics]
 ↪  Driver: i915
            Card: NVIDIA Corporation AD107GLM [RTX 2000 Ada Generation
             ↪  Laptop GPU]  Driver: nvidia
Audio:      Card: sof-soundwire  Driver: snd_soc_sof_sdw
            Server: PipeWire  v: 1.6.2  ALSA: k7.0.0-29-generic
Sched:      policy: SCHED_OTHER  nice: 0  (real-time available up to 50, not
 ↪  used)
Displays:   [0] eDP-1                  2560x1600  60.040 Hz  bounds 0,0
 ↪  2560x1600  [primary]
            [1] DP-5                   2560x1440  59.950 Hz  bounds 2560,0
             ↪  2560x1440
            video driver: kmsdrm   (the [N] above is the -d N value)
Audio out:  [0] Dummy Output
            driver: pulseaudio
```

Reviewers routinely ask for CPU model, OS version, graphics driver and audio server. Two lines repay a second look:

- **`Sched:`** records the policy the run actually got, not the one it asked for. See Setting priority under Linux.
- **`Displays:`** — on a multi-monitor machine the heads differ in resolution, refresh rate and pixel density, and every calibration is per-display. Each data file's `-`

`info.txt` records **which display the window actually opened on**; read it from there rather than assuming.

`-sysinfo` also prints a `Vblank:` line. By default, onsets are timestamped when `SDL_RenderPresent` returns; `GOXPY_VBLANK=on` instead anchors every frame on the kernel's DRM vblank stamps. **It is off by default and there is currently no reason to turn it on** — measured against a photodiode the two are indistinguishable, both flat to well under a ppm. The case it exists for is a host whose nominal refresh rate is badly wrong. If you do enable it, read `sys vblank_resolution:` in the run's `-info.txt`: **if it ever says `WRONG DISPLAY`, the onsets in that file are on another monitor's frame grid and must not be used.** The mechanism lives in the `vblank/` package.

The `Frame pacing` block

`-test display` and `-test av` both end with a block like this:

```
── Frame pacing ──────────────────────────────
  presents: 588   (frame = 16.656 ms)
  blocked : 566 (96.3 %)  — present carried the frame; its return is the
   ↪  onset
            of which 281 came back inside the nominal boundary by mean
            0.285 ms, max 1.861 ms — the phase offset between the nominal
            frame grid and the panel's, not a wait.
  paced   : 22 (3.7 %)  — returned early; Update held to the schedule
            wait mean 2.496 ms  max 2.696 ms
  verdict : the driver blocks. Flip timestamps carry the display's own
            instant, and cannot drift against the panel.
```

It does not report dropped or missed frames. Every present is counted here; a frame that never reached the panel shows up in the interval statistics above it, not in this block.

It answers a different question: *what were this run's flip timestamps anchored to?* `SDL_RenderPresent` is supposed to block until the retrace, but under triple or mailbox buffering it queues the frame and returns immediately. When it does, `Screen.Update` holds the frame to the boundary itself — and the onset it reports is then a *scheduled* time rather than a hardware one.

line	meaning	the onset `FlipTS` reports
`blocked`	present covered the frame and returned at, or just inside, the boundary	the present's own return — a hardware instant
`early`	the part of `blocked` that returned *just* inside the nominal boundary	unchanged: still the present's return
`paced`	present came back with most of the frame left, so `Update` held it	the scheduled boundary — synthesised

line	meaning	the onset `FlipTS` reports
`vblank`	a kernel vblank timestamp was available (`GOXPY_VBLANK=on`)	the vblank — measured

A large `early` count is not a warning. A blocking present returns one *panel* period after the last one, while the boundary it is compared against is one *nominal* period after — and the two grids are never in exact phase. It is a constant offset, re-established by the hardware every frame, so it cannot accumulate, and it cancels out of any duration or reaction time (both are *differences* between timestamps).

Read the **verdict**, not the percentages: it is computed from the time held per present across the whole run, precisely because a driver that blocks but sits a fraction of a millisecond off the nominal grid can take the held branch on nearly every frame while blocking for 96 % of each one.

- **`the driver blocks`** — nothing to do. Onsets are hardware-anchored.
- **`onsets come from the kernel's vblank timestamp`** — nothing to do; the most accurate configuration available.
- **`the driver does NOT block`** — your frames are still correctly paced and your *durations* are fine, but the onsets are stamped with a schedule running at the nominal refresh rate. If that rate is wrong, absolute onsets slide against the panel over long blocks. Compare the estimated refresh rate printed above against the nominal one — the gap between them *is* the drift rate — and run `timing-drift` on a photodiode capture before quoting absolute onsets.
- **`MIXED`** — some frames took each path. Worth a photodiode check before quoting absolute onsets.

A mostly-paced machine is not broken, and that is exactly the problem: **pacing makes a non-blocking driver produce software statistics indistinguishable from a blocking one.** What it cannot do is keep the schedule and the panel in step forever.

Only the third verdict asks anything of you, and even then it bounds a *slow slide* over minutes, not per-trial error.

The same counts are written to every run's `-info.txt`, so you can tell after the fact which branch a capture took:

```
# sys pacing: presents=30000 blocked=30000 early=29610 paced=0
 ↪  vblank_held=0 (0.0 % paced) … class=blocking
# sys pacing: presents=30000 blocked=0 early=0 paced=30000 (100.0 % paced)
 ↪  wait_mean=16.141 ms … class=not-blocking
```

A drift-free result means two different things in those two cases. `class=blocking` means every frame re-anchored to the hardware, so the run says little about how accurate the pacing schedule is. `class=not-blocking` means the schedule *was* what advanced the clock, and a flat result there is evidence about the schedule itself. Without this line the two are indistinguishable afterwards, and a set of captures can end up unable to answer the question it was run for.

`av` — the stimulus timing test

This is the one that matters. Every cycle presents all three modalities at once:

- **Visual** — five squares (four corners + centre) bright for `-frames-on` frames, then dark for `-frames-off` frames
- **Audio** — a tone lasting `frames-on × frame-period`, synchronised to the visual onset or offset from it by `-soa-ms`
- **TTL** — a `-trigger-ms` pulse at the visual onset, on the device chosen by `-trigger-device`

```
Timing-Tests -test av -cycles 100          # 200 ms on, 300 ms off at 60 Hz
Timing-Tests -test av -no-sound -no-ttl    # visual only, no hardware at all
Timing-Tests -test av -soa-ms -50         # audio leading the flash by 50 ms
```

The frame period is read from the display, not passed in; `-frames-on 12` is 200 ms only if the panel really runs at 60 Hz, which is what `-test display` tells you.

`-no-sound` and `-no-ttl` drop a modality, which is how one test covers what used to be three.

What to record. A photodiode on one square, the TTL line on a second channel, a microphone if you are checking audio. The quantity of interest is the *offset* between the TTL edge and the luminance step, and how stable it is across cycles. A constant offset is a calibration you subtract; a varying one is jitter you cannot.

What the console tells you. Bright-phase duration, cycle period, and (with sound) audio-queued minus visual onset. All software-side — cross-checks, not ground truth.

Expect a warm-up transient. The first several cycles run long and variable, and they dominate the summary statistics if left in. `-warmup` (default 10) excludes them from the test's own statistics, but offline analysis tools do not know about it — quote steady-state numbers, and say which cycles you used.

Audio onsets are quantised by the buffer. With `-audio-frames 256` at 44100 Hz, tone onsets land on 5.8 ms steps. This appears as a lattice in the inter-onset intervals and is not jitter — it is the buffer. See Audio: the buffer size is the whole story before choosing a value; smaller is emphatically not better.

Choosing a TTL output device. `-trigger-device` selects among `dlpio8` (USB serial, 5 V), `parallel` (ppdev `ioctl`, 5 V), `gpio` (GPIO chip `ioctl`, 3.3 V), `ft232h` (USB bulk, 3.3 V) and `labjackt4` (Modbus TCP, 3.3 V). They are not interchangeable — the trigger fires immediately after the flip returns, so whatever the write costs lands in the trial-to-trial *spread* of the flip-to-edge interval. The full table, the per-device meaning of `-trigger-pin`, and the Linux prerequisites are in `tests/Timing-Tests/README.md`, under "Choosing a TTL output device"; what the differences actually cost is in How much the trigger device costs.

> **A device that fails to open no longer silently continues.** `av` refuses to start if the chosen trigger device could not be opened, unless you passed `-no-ttl`. Still watch the start-up lines — they echo the device, the resolved pin and the pulse width.

Running a whole session. `tests/Timing-Tests/run-timing-tests.sh` drives the standard sequence (`sysinfo check display display-gc av av-gc av-visual latency`) and can wrap the photodiode steps in a `bbtk-capture` recording so the stimulus always falls inside the capture window. Set `BBTK_CAPTURE=1` and `BBTK_CAPTURE_BIN`; `BBTK_MARGIN_S` (default 8) sets how much is recorded either side. Its environment variables are documented in its own header and in `tests/Timing-Tests/README.md`.

If you are recording audio with a BBTK microphone, max the output volume and place the microphone within a few millimetres of the speaker membrane. Anything less and events are silently dropped. Two checks on the resulting `-events.csv`: `N(Mic1)` should equal `N(TTLin1)`, and the Mic duration should be close to `frames-on × frame period`. A 197-cycle run yielding 6 Mic events is a placement problem, not a timing one.

`vrr`: arbitrary stimulus durations

On a 60 Hz monitor every duration is a multiple of 16.67 ms. At 120 Hz the quantum shrinks to 8.33 ms — better, but still coarse for subliminal work where you may want 10, 12 or 15 ms.

Variable Refresh Rate — AMD FreeSync, NVIDIA G-Sync, or the underlying VESA Adaptive-Sync standard — removes the quantisation: the display holds the current frame for exactly as long as you ask. Durations become controlled by your software timer rather than the display clock.

```
Timing-Tests -test vrr -vrr-max-ms 50 -vrr-reps 5
```

The test switches the renderer to vsync=0 for its duration (restored on exit), sweeps target durations from 1 ms to `-vrr-max-ms` (default 20) in 1 ms steps, `-vrr-reps` times each, holding with a busy-wait and recording what was achieved. It prints one line per target plus an aggregate over the sweep.

- **On a working VRR display:** duration error stays small and roughly constant across the whole sweep.
- **On a fixed-refresh display:** the error is *periodic* — near zero at multiples of the frame period, rising in between. A sawtooth means you do not have VRR, whatever the monitor's box claimed.

On Linux, check the connector first (`sudo cat /sys/kernel/debug/dri/0/*/vrr_capable`, looking for `vrr_capable: 1`) — then confirm with the test rather than trusting the setting.

We have no photodiode capture of a VRR run. The console numbers describe what the busy-wait achieved, not what the panel did.

`rt` — reaction-time timestamp precision

```
Timing-Tests -test rt -cycles 50
```

Flashes the screen at a jittered interval and waits for a key press, recording the SDL3

hardware event timestamp against the flip timestamp. This characterises the *input* path: how much delay and jitter your keyboard, USB stack and event queue add.

Press as fast as you can, or better, use a solenoid — you are characterising the machine, and human variability swamps it. The useful number is the SD, not the mean.

`timing-drift`: is the flip timestamp tracking the panel?

The two `av` output files each look fine on their own; a drift failure exists only *between* them. This tool joins them and reports the slope first:

```
make timing-drift
./_build/timing-drift bbtk-av-001-events.csv Timing-Tests_sub-000_
 ↪  date-….csv
```

Options: `-ttl` (channel name, default `TTLin1`), `-light` (default `Opto1`), `-hz` (0 = read the run's sibling `-info.txt`), `-warmup` (10), `-out`.

It prints, for TTL→light and then for flip-timestamp→light:

- the **slope**, in µs/cycle and ppm — the number that matters;
- the **de-trended SD**, the real trial-to-trial scatter once the ramp is gone;
- how much of the variance the ramp accounts for;
- one-frame jumps, and the panel's **true frame period** fitted from the photon train, scored against both rates the framework recorded.

The host and the instrument run off different crystals — tens of ppm apart before anything interesting has happened — so the tool fits that clock ratio out of the trigger-versus-flip relation rather than assuming it away. Events are paired by nearest neighbour within half a cycle, not by index, because a single spurious detection shifts every later pair by a whole cycle.

Every slope is fitted over the longest stretch containing no dropped frame, and the `fitted over` line says which cycles those were. This is not tidying: a frame-sized step at index k of n points biases a least-squares slope by $6 \cdot h \cdot m \cdot k/n^3$ (with $m = n-k$), which at the midpoint is $1.5 \cdot h/n$. Over a 1000-cycle run at 60 Hz that is 25 µs/cycle — **50 ppm from a single dropped frame in eight minutes**, ten times the effect these captures exist to resolve, and it does not average away with more cycles because the lever arm grows with n too. On a short capture the bias is larger still: 20 cycles with one drop puts it above 2000 ppm.

The `one-frame jumps` count is printed separately, and a run with more than a few of them is not a usable capture however clean the surviving stretch looks.

Read the verdict line. `DRIFT DOMINATES` means the flip timestamps and the panel are on different clocks and no absolute onset from that run can be trusted; `NO DRIFT, but … scatter` means the timestamps track the panel and the spread is genuine.

Rules of thumb for your own numbers

These thresholds are judgement, not measurement. No capture in this repository established them; they are a starting point for a reader with no reference at all.

For real reference points — machines, dates, sample sizes — use Part 2.

Metric	Excellent	Acceptable	Problematic
Frame-interval SD (`display`)	< 0.1 ms	< 0.5 ms	> 1 ms
Frames > 0.5 ms late (`display`)	< 1 %	< 5 %	> 10 %
Audio pipeline latency (`latency`)	< 15 ms	< 30 ms	> 50 ms
Bright-phase duration SD (`av`)	< 0.2 ms	< 1 ms	> 2 ms
TTL→photodiode SD (`av`, hardware)	< 0.5 ms	< 2 ms	> 5 ms
Dropped frames per 100 cycles (`av`)	0	< 5	> 10
VRR duration error SD (`vrr`)	< 0.1 ms	< 0.5 ms	> 1 ms (or periodic → no VRR)
RT SD (`rt`)	< 3 ms	< 10 ms	> 20 ms
Pacing verdict (`display`)	vblank, or blocks	MIXED	does NOT block

The hardware rows are the ones worth quoting in a methods section.

The pacing row grades *what the onsets are anchored to*, not frame quality, and it is the one row where "problematic" still leaves durations and reaction times intact — read The `Frame pacing` block before acting on it.

An SD alone cannot grade a TTL→photodiode series, and that row is the one most likely to mislead. A delay that slides steadily across a run has an SD like any other, and a large one — yet nothing about it is jitter, it cannot be averaged away, and every within-channel statistic in a BBTK report looks perfect while it happens. Measured on a Raspberry Pi 4: SD 4.07 ms, of which **99.8 % was a monotonic 13.9 ms ramp** and 0.18 ms was real scatter. Run `timing-drift` before reading this row.

Loading the data in Python

Each run writes two files to `~/goxpy_data/`, or to `-outdir` if given:

- `Timing-Tests_sub-000_date-<YYYYMMDD>-<HHMMSS>.csv` — data rows, no comments
- `...-info.txt` — session metadata: times, host, OS, and the display and audio configuration actually in use

```
import pandas as pd

df = pd.read_csv("~/goxpy_data/Timing-Tests_sub-000_
 ↪  date-20260817-091541.csv")
```

```
# av: drop the warm-up cycles before quoting anything
steady = df[df.cycle >= 10]
print(steady.bright_duration_ms.describe())
print(steady.period_ms.std())
```

Every CSV starts with `subject_id`. After that:

Test	Columns
`av`	`cycle`, `t_visual_before_ms`, `t_visual_after_ms`, `bright_duration_ms`, `period_ms`, `t_audio_queued_ms`, `soa_intended_ms`, `soa_actual_ms`, `onset_source`
`display`	`frame`, `t_before_ms`, `t_after_ms`, `interval_ms`
`rt`	`trial`, `onset_ns`, `event_ts_ns`, `rt_ns`, `rt_ms`
`vrr`	`target_ms`, `rep`, `actual_ms`, `duration_error_ms`, `onset_ns`, `offset_ns`, `trigger`
`latency`	`duration_ms`, `rep`, `drain_ms`, `overhead_ms`
`check`	none — the CSV is empty; only the `-info.txt` is meaningful

`onset_source` records how each onset was anchored: `hardware-verified`, `present-return`, `pacing-schedule`, `vsync-estimated` or `unknown`. Captures made before this column existed do not have it.

Always read display parameters from the run's own `-info.txt` rather than assuming — on a multi-monitor machine the heads differ in every relevant parameter, and the header records which one the window opened on.

Improving timing on your system

If you need a sub-millisecond spec, in order of what it is worth. Every figure is measured on this page or in Minimising trigger jitter:

#	do this	worth
1	Get off the compositor — `SDL_VIDEODRIVER=kmsdrm` from a VT, or a non-compositing WM with exclusive fullscreen	**16×** (sd 1.344 → 0.083–0.113 ms)
2	Run at `SCHED_FIFO` 50	**1.8×** (sd 2.342 → 1.320 ms)
3	CPU governor `performance`	drift +11.8 → **+0.41 ppm**

#	do this	worth
4	Count frames, never sleep, for stimulus durations and ISIs (`ShowFrames`/`BlankFrames`)	trigger→light spread 14.1 → **3.4 ms**
5	Raise the trigger synchronously on the flip thread (`triggers.FireTriggerSync`), not `go FireTrigger(...)`	0.73 ms → **34 µs**
6	Discard warm-up trials — the display settles	sd 1.127 → **0.384 ms** over the first 10
7	Measure the flip→photons offset once on this rig, record it, subtract it in analysis	it is constant to 83–113 µs; see below

Steps 1–3 are the display and the scheduler, 4–6 are how the experiment is written, and 7 is the one thing no setting can remove.

Linux

- **Get off the compositor.** By far the largest single improvement, and it is not close — see Part 2. Start from a virtual terminal (`Ctrl+Alt+F2`) with the window system stopped (`systemctl stop gdm`) and `SDL_VIDEODRIVER=kmsdrm`. Failing that, a plain non-compositing window manager (openbox, i3) with exclusive fullscreen is the other measured-good configuration. See Linux virtual console for the SDL driver selection details.
- **Run at real-time scheduling.** Worth a measured factor of 1.8 in onset jitter. Grant your user the privilege once; after that goxpyriment programs — `Timing-Tests` included — request priority 50 at startup on their own:

  ```
  Timing-Tests -test display -duration-s 30    # asks for real-time itself
  Timing-Tests -realtime-priority 0 -test display   # opt out
  ```

  The `Sched:` line in the system report records which policy it actually got. The full procedure — and the traps, including that the limit is inherited from your login session and a new terminal will not pick it up — is in Setting priority under Linux.
- Disable CPU frequency scaling: `cpupower frequency-set -g performance`.
- Consider a real-time kernel — see https://ubuntu.com/blog/enable-real-time-ubuntu.
- **Send triggers off the USB bus if you can.** `-trigger-device parallel` on a machine with an LPT port, or `-trigger-device gpio` on a single-board computer; both write via one `ioctl`. Before assuming `/dev/gpiochip0` owns the header, ask the system (`gpiodetect`, `gpioinfo`) — which chip carries the 40-pin header varies

by board and kernel, and another chip's lines may drive real board hardware. Verify the wiring with `tests/test_linuxgpio` or `tests/test_parallel_port` before spending a long capture.

- **Lower the FTDI latency timer** if you *read* responses over a USB serial device. It does nothing for *sending* triggers — see the DLP-IO8 appendix.

Windows

- Disable "Hardware-Accelerated GPU Scheduling" in Display Settings if you see high frame jitter. See Setting priority under Windows.

Part 2 — What we found

Every figure below names the machine, the stack, the date and n. The full workings live in Minimising trigger-to-stimulus jitter and the mega-study comparison; this is the summary.

The display stack is the dominant term

Three runs differing only in the display stack — same binary, same machine, same panel, same night. Dell Precision 5490, Intel Arc (Meteor Lake), 60.04 Hz, `SCHED_FIFO 50`, synchronous trigger, DLP-IO8 and photodiode captured in one Analog Discovery 3 acquisition at 200 kS/s so the instrument's clock cancels. Onset is the photodiode's 10 % crossing. **2026-08-08**, ~590 trials per arm:

stack	mean	**sd**	full range	flips > 1 ms off the frame grid
Wayland session	21.75 ms	1.344 ms	18.83–36.74	7.1 %
KMS/DRM, no display server	18.91 ms	**0.113 ms**	18.58–19.13	**0 of 590**
Bare Xorg + openbox, exclusive fullscreen	35.74 ms	**0.083 ms**	**35.52–35.95**	**0 of 580**

Sixteen times better, and the mechanism is gone rather than reduced. Off a compositor, not one flip in ~590 lands more than 1 ms off the frame grid; under Wayland one trial in fifteen does. An entire five-minute Xorg run fits inside a 0.43 ms band. By comparison, real-time priority on the same rig is worth 1.8× (sd 2.342 → 1.320 ms).

The result is three-way, not "kmsdrm beats Wayland". Bare Xorg is the *steadiest* and the *latest*, which is not a contradiction — they are different quantities. The gap is exactly one frame:

`Xorg − KMS/DRM = 16.826 ms = 1.010 frames`

One more buffer in the pipeline, dead constant. For EEG and MEG the distinction is the whole point: **a constant 19 or 36 ms is subtracted in analysis and costs nothing; 1.3 ms of scatter around it cannot be.** Measure your offset once per rig, record it, subtract it.

Set against the fourteen lab configurations of Bridges et al. (2020) Table 2, the bare-Xorg run has the best onset precision in the table by a factor of two, and the Wayland run is worse than thirteen of the fourteen — *the same binary on the same machine*. See the mega-study comparison.

The display stack is worth two frames of latency and five times the jitter A second, independent campaign found the same effect on different hardware. **2026-08-17**, BBTK v3, a Radeon Pro W5700 (radeonsi) driving a DLP-IO8, one variable changed:

W5700, everything else identical	onset latency	scatter	n
X11, exclusive fullscreen	54.88 ms	0.296 ms	481
kmsdrm, bare console	**22.55 ms**	**0.057 ms**	286

32.4 ms is almost exactly two frames — an X11 Present path plus a driver swapchain queueing ahead, where on kmsdrm the application page-flips straight to the CRTC.

This is a different machine and a different campaign from the table above, which is why its X11 scatter (0.296 ms) is not the 0.083 ms measured on the laptop's bare Xorg. That configuration was openbox with no compositor at all; this one was a full X session. The two are not the same condition, and the comparison to draw from each is the within-campaign one.

The panel is not involved: its 10–90 % transition measured 16.29 ms on a Pi 4 and 17.28 ms here, with the same asymmetry between halves, so the same monitor behaves the same way on both machines and the latency is upstream of it.

None of this is visible from inside the program. Both configurations report `class=blocking`, sub-ppm drift against the panel, and frame intervals with sub-millisecond scatter. An experiment quoting an absolute onset would be 32 ms wrong under X11 with no way to know.

What a returning `SDL_RenderPresent` actually means

The tables above raise an obvious question: if the present blocks on VSYNC, why are the photons one to three frames later?

Because it does not block on *your* frame. `SDL_RenderPresent` returns when the driver will accept the **next** frame — i.e. when a buffer slot frees, which happens as an *earlier* page flip completes. The frame just drawn becomes the scanout source at a later vblank. This is true even on bare KMS/DRM with no display server; it is a property of the swap chain, not of the compositor.

Psychtoolbox gets scanout semantics deliberately: it schedules the swap against a specific future vblank with `glXSwapBuffersMscOML`, then blocks in `glXWaitForSbcOML` until *that* swap has completed. Hence its 4.53 ms in Bridges et al. (2020) against our 18.91 ms on the equivalent stack.

This also explains an apparent contradiction in the numbers above. The pacing block reports `class=blocking`, with present covering 15.99 ms of a 16.66 ms frame. That is real — it is simply blocking on the *previous* frame's completion. Nothing inside the process can tell the two cases apart, which is why an experiment quoting an absolute onset can be 32 ms wrong with no indication.

So a correct cadence and a multi-frame absolute offset are not in conflict. The flip timestamps track the panel to sub-ppm (next section) *and* lead the photons by one to three frames. The first makes durations and intervals exact; the second is a constant that has to be measured per rig and subtracted. Both are true at once, and only the second needs a photodiode.

The arithmetic closes: subtracting the queue depth from the archived flip→photons figures leaves 4.9–6.0 ms on one panel across two machines and two stacks — inside the 2.35–7.10 ms band Bridges et al. report for every Linux and Windows package they tested. The workings, and the limitation that the diode position was not recorded, are in `TODO.md` under "Presentation latency".

Flip timestamps track the panel

Two machines, BBTK v3 photodiode, **1010 cycles each, 2026-08-17**, after the flip-timestamp anchoring was corrected:

	Raspberry Pi 4 (V3D/kmsdrm)	Radeon Pro W5700 (radeonsi/X11)
flip → photons, slope	**+0.01 ppm**	**+0.13 ppm**
total drift over the run	0.006 ms / 8.3 min	0.066 ms / 8.3 min
de-trended scatter	0.128 ms	0.278 ms
one-frame jumps	0	0
TTL → light	23.43 ms, sd 0.075 ms (GPIO, kmsdrm)	54.88 ms, sd 0.296 ms (DLP-IO8, X11)

The W5700 is the informative case for drift. Its GPU and CPU keep independent crystals, so its loop cadence runs **−4.20 ppm** off nominal; before the anchoring was corrected, its flip timestamps advanced on the CPU clock at the nominal rate while the panel ran on the GPU clock, and that difference showed up as a measured **−4.73 ppm** of drift. The two numbers are the same quantity. That the cadence is still −4.20 ppm while the drift is now +0.13 ppm is the evidence that onsets follow the panel rather than the schedule.

For a sense of what a healthy `display` run looks like on a good stack, the laptop above on kmsdrm (2026-08-09, `tests/Timing-Tests/report-dell-precision-5490-laptop-kmsdrm/display-gc-off.txt`) estimated 60.037 Hz with frame intervals **mean 16.656 ms, SD 0.008 ms** over n=677, and not one interval more than 0.5 ms late.

Audio: the buffer size is the whole story

Measured on a Raspberry Pi 4 (PipeWire, 48 kHz) on **2026-08-17**, capturing the line output directly with an Analog Discovery 3 at 100 kS/s and taking the onset from a running-RMS envelope:

driver	`-audio-frames`	period	n	median AV lag	SD	`period/√12`	torn tones
PipeWire	2048	42.67 ms	60	101.50 ms	12.28 ms	12.32 ms	none
PipeWire	**1024**	21.33 ms	481	**49.88 ms**	**6.17 ms**	6.16 ms	**none**
PipeWire	512	10.67 ms	500	27.30 ms	7.28 ms*	3.08 ms	10 (2.0 %)
PipeWire	256	5.33 ms	483	9.43 ms	2.36 ms	1.54 ms	100 (20.7 %)
ALSA direct	512	10.67 ms	463	4.95 ms	6.74 ms	3.08 ms	**463 (100 %)**

* inflated by a mid-run escalation, below; its first half scatters by 3.10 ms.

1024 is the recommended setting on this hardware — the only one that is both clean and stable, and what `run-timing-tests.sh` now uses.

A large buffer costs jitter, and the jitter is a beat, not noise. The tone is handed over on time and then waits for the queue, so the audio onset scatters across one buffer period — matching `period/√12` to within 0.2 % at 1024 and 2048. But it is deterministic: the tone's position in the buffer grid advances by `cycle mod period` every trial and wraps. At 512, a 499.830 ms cycle predicts +1.503 ms per trial with a −9.163 ms wrap; observed, +1.500 ms (n=161) and −9.170 ms (n=58) — agreement to 7 µs. So a per-trial lag can be **predicted** rather than merely bounded, and a cycle chosen as a whole multiple of the buffer period removes the beat altogether.

A small buffer costs glitches, and PipeWire moves the goalposts. At 512, ten of the first 245 tones were torn (gaps to 37.7 ms); then at trial 245 the lag stepped by **+10.85 ms — one buffer period — and the tearing stopped dead**, with no glitch in the remaining 255 tones. That is the audio server adding a period to the graph after repeated underruns. A naive linear fit through that step reports a 15 ms "drift" that does not exist.

The floor is the hardware, not the audio server. Bypassing PipeWire entirely (`SDL_AUDIODRIVER=alsa` on `hw:2,0`, server stopped) gave the lowest latency of anything measured — median 4.95 ms, some trials negative — and was completely unusable: **every one of 463 tones was torn**, typically five times each. So the Pi cannot sustain a 10.7 ms buffer, and PipeWire's mid-run escalation was it discovering the same floor and working around it. Sit above the floor deliberately rather than removing the thing that was hiding it.

Scheduling priority is not involved. The 2×2 of 512/2048 against `-realtime-priority 50/0` scratched at 512 under both policies and was clean at 2048 under

both.

None of this appears in what the host prints. In these same runs the software-side SOA read **0.080 ± 0.035 ms** — handing the tone over on time is the part the software does correctly. Check underruns at the source instead: `pw-top` over ssh, watching the **ERR** column.

One methodological note worth keeping. The 512 case was first measured through a BBTK microphone channel, which reported 23 % of tones torn by gaps whose median was 22.3 ms; the electrical capture found the tearing real but affecting 2.0 % of tones, mostly by about 2 ms. A threshold detector watching an acoustic envelope needs the signal to recover past its threshold before it calls the tone present again, so short electrical glitches read as long acoustic gaps. **A second instrument measuring by a different route is the only way to learn what the first one is adding** — and prefer the route with fewer transducers when the question is about software.

How much the trigger device costs

Two campaigns on **2026-08-21** put trigger devices on one schedule and one timebase, an Analog Discovery 3 at 1 MS/s:

comparison	machine	result
DLP-IO8 vs FT232H	Precision 5490, `SCHED_FIFO 50`, 1.5 V threshold	DLP-IO8 lags by **179.4 µs**, sd 5.7, n=322 over three 60 s runs
DLP-IO8 vs parallel port	Precision 3650 Tower, 2.5 V threshold	DLP-IO8 follows by **185.9 µs**, sd 12.6, n=900; worst pulse 254.8 µs

The host's own contribution — flip return to trigger write — was **0.03 µs median**. So the ~180 µs is the USB serial path, it is consistent across two machines and two comparison devices, and against a 1 ms trigger budget it leaves a 3.9× margin.

The practical reading: the DLP-IO8 is fine for a millisecond-scale budget, and the `ioctl` paths (`parallel`, `gpio`) are measurably tighter if you have them. Full data in `tests/test_triggers/report-is158520/` and `tests/test_triggers/report-precision3650/`.

What we have not measured

Stated so the next reader knows where the edges are:

- **The browser.** WASM frame pacing has been characterised in software (16.666 ms, SD 0.11–0.12 ms over 299 frames, headless Chrome, 2026-07-13) but there is **no photodiode validation of actual pixel onset** — see WASM.md.
- **VRR.** No photodiode capture of a `vrr` sweep exists. The console numbers describe the busy-wait, not the panel.
- **Reaction time.** No solenoid or hardware-actuator characterisation of the `rt` input path.

- **Scan-out gradient.** No current report directory carries a top-to-bottom gradient measurement, which is why Part 1 tells you to measure your own.
- **macOS and Windows.** Every figure on this page is from Linux.

Related tests

- `tests/test_gv_sync` — `.gv` video playback: does `PlayGvFunc` put a frame on screen when it says it does?
- `tests/test_linuxgpio` — GPIO character-device output/input, used to verify header wiring before a session.
- `tests/test_parallel_port` — LPT data lines via ppdev, the equivalent check for a parallel-port trigger.
- `tests/test_dlpio8` — square-wave output for characterising the trigger box itself against an oscilloscope.
- `tests/test_triggers` — two trigger devices against each other on one timebase; the source of the 180 µs figure above.
- `tests/test_clear_only_frames` — regression test for the compositor presentation bug described at the top of this document.
- `tests/test_vsync_blocking` — does `SDL_RenderPresent` block on VSYNC on this platform, or return immediately?

Contributing a report

If you measure your own rig, we would like to see it — good results help people choose hardware, and bad ones tell us what to fix. See `performance-reports/README.md` for what to include.